ImportSafe

Frontend technical-round guide

Understand the code. Explain the choices. Know the limits.

Prepared for the HackIIITD team | 24 October 2026

This guide explains the frontend currently in:

`C:\PC\Projects\IIITD\import safe\frontend`

It is based on the active source code and package configuration, rather than the previous frontend backup. Read the answer scripts aloud, then practise explaining them in your own words.

Your 45-second introduction

“ImportSafe helps Indian buyers organise the questions they need to answer before importing a product. Our frontend is a React application built with Vite, TypeScript and JavaScript, custom CSS, and Framer Motion. It includes a public landing site, demo login and sign-up, an import-details form, a six-stage investigation preview, and a detailed report with editable cost assumptions and an evidence drawer. The current frontend is a working interaction prototype. Live research and production authentication still need to be connected.”

How to study this guide

- **First 15 minutes:** pages 2-4, 12, and 16-18. Learn the stack, the file map, YAML versus JSON, and the interview answers.
- **Next 30 minutes:** pages 5-11 and 13-15. Understand how the implementation works and where the boundaries are.
- **Before presenting:** page 19. Rehearse the demo and the questions that expose unsupported claims.

The most important rule

Explain the product vision separately from the code that runs today. A precise explanation of a prototype is stronger than claiming that a timed animation performs real research.

01 / What is actually implemented?

Working frontend behaviour

- Landing page plus separate Features, How it works, Architecture and Pricing views.
- Animated transitions to login/sign-up, including a paper-plane path.
- Demo login using `demo123`; sign-up checks matching passwords and basic input constraints.
- Product-link, quantity, destination, purpose, shipping and certificate inputs.
- Six timed investigation stages, cancel action, and manual product entry for custom listings.
- Report sections, editable illustrative costs, checklist, evidence drawer, seller-message copy, JSON export, and browser print/PDF.

Boundaries you must explain

Topic	Current truth
Authentication	A browser-state demo. No account creation, server session, password hashing or durable login.
Research	The active frontend does not call a research API. Timers advance the preview.
TinyFish	Shown in the intended architecture; not called by the active frontend.
Verification	Product documents and official records are not independently retrieved.
Costs	Local calculations with editable sample assumptions; not live duty or FX data.
Storage	Component memory. Reloading clears the demo sign-in and workspace state.
Backend	A separate project folder exists. The replacement frontend is not wired to its API hooks.

Safe wording in the round

“We implemented the frontend interactions and report presentation. The investigation service is an integration boundary. Until it supplies verified results, the UI labels the report as a demo and keeps the outcome at insufficient information.”

Do not describe the login as secure production authentication, the timeline as live scraping, or the source links as evidence that was fetched. Build success proves compilation, not business accuracy or security.

Source basis: `main.tsx`, `Login.jsx`, `Investigation.tsx`, and `InvestigationFlow.tsx` in the active frontend.

02 / The frontend technology stack

“Stack” means the set of languages, libraries and tools used together to build the app.

Technology	What it does in our project	Why it fits
HTML	Provides the page entry and semantic elements such as forms, headings and buttons.	Browser-native structure and interaction.
CSS	Controls the sage/cream theme, layouts, responsiveness, grain, glass effects and keyframes.	Custom styling without a component UI kit.
JavaScript	Runs browser behaviour and the `.jsx` login component.	The browser's application language.
TypeScript	Adds types in `.tsx` components such as orders, props and state.	Catches many mistakes before runtime.
React + React DOM	Compose the UI from components and render it into the page.	Reusable views and state-driven updates.
Vite	Runs the local development server and produces the production build.	Fast feedback and a simple build workflow.
Framer Motion	Drives card transitions, motion values, springs and the navigation plane.	Animation close to React component logic.
SVG	Draws the paper planes and illustrations as vectors.	Scales clearly at different screen sizes.
Node.js + npm	Run development tools and manage packages.	Standard JavaScript tooling.

Versions: explain ranges correctly

The manifest declares React/React DOM `^19.0.0`, Framer Motion `^14.6.0`, Vite `^7.0.0`, TypeScript `~5.9.3`, and React type definitions `^19.0.0`. These are dependency ranges. The lockfile records the resolved dependency tree; a range is not necessarily the exact installed version.

The reviewed lockfile resolves React/React DOM 19.3.1, Framer Motion 14.6.0, Vite 7.3.7 and TypeScript 5.9.3. Recheck after dependency updates.

The active replacement does not use Tailwind, Bootstrap, Material UI, Redux, React Router or React Query. Plain CSS and a small custom hash router are used. Those packages or files may appear in the previous frontend backup, which is a different implementation.

References: React UI concepts, Vite, Motion documentation.

03 / Where everything lives

File or folder	Responsibility
`index.html`	Contains the root element and loads `src/main.tsx`.
`src/main.tsx`	Mounts React, selects the current view from the hash, and owns demo sign-in state.
`Landing.tsx` / `.css`	Home content, illustrations, calls to action and layout.
`PublicPages.tsx` / `.css`	Features, How it works, Architecture and Pricing; shared public shell.
`Login.jsx` / `.css`	Login/sign-up modes, fields, validation feedback and background planes.
`PageFlight.tsx` / `.css`	Intercepts suitable auth links and animates a plane before navigation.
`Investigation.tsx` / `.css`	Collects import details, handles demo prefill and starts the local workflow.
`InvestigationFlow.tsx` / `.css`	Progress stages, manual fallback, report, calculator, dialogs and export.
`style.css`	Shared baseline styles. Page-specific styles are defined in the other CSS files.
`package.json`	Dependencies and development/build scripts.
`package-lock.json`	Resolved package versions and dependency integrity information.
`tsconfig.json`	TypeScript checking, JSX and module-resolution settings.
`node_modules/`	Installed packages. Generated; do not edit it as application source.
`dist/`	Production build output. Generated; deploy this output rather than the source tree.

Follow a user action through the code

`index.html -> main.tsx -> Landing -> PageFlight -> Login -> Investigation -> InvestigationFlow -> report`

The report is a view inside `InvestigationFlow.tsx`; it is not a separate backend-generated page. The progress and report remain under `#analyze` while React state changes which view is visible.

Routes you should recognise

`#home`, `#features`, `#how-it-works`, `#architecture`, `#pricing`, `#login`, `#sign-up`, and `#analyze`.

The # part is the URL fragment. It is normally handled by the browser rather than sent to the server as part of the HTTP request target.

04 / HTML, CSS, JavaScript and TypeScript

HTML: what is on the page

HTML describes structure and meaning. A heading is an `<h1>`, a form is a `<form>`, and a clickable action should usually be a `<button>`. Semantic elements help keyboard users, screen readers and browser behaviour.

CSS: how it looks and adapts

CSS chooses colours, spacing, font sizes and layout. The box model consists of content, padding, border and margin. Flexbox is useful for arranging a row or column; Grid is useful for arranging rows and columns together. Media queries change layout according to conditions such as viewport width.

Our design uses custom properties such as `--cream` and `--sage-deep`, plus page-specific tokens. A custom property is a reusable CSS value; it is not a JavaScript variable. Fraunces is used for headings and Inter for body text.

JavaScript: what happens when the user acts

JavaScript handles events, updates state, calculates costs, manages timers and calls browser APIs. An event handler is a function called after an action such as clicking or typing. A callback is a function passed to another function to be called later.

TypeScript: useful checks before the app runs

TypeScript adds a type system to JavaScript. For example, an order can declare `quantity: string` and `certificates: string[]`. The compiler can then catch passing an incompatible value. Types are removed from the generated JavaScript; they do not validate an untrusted API response at runtime.

What are JSX and TSX?

JSX is syntax that lets us describe UI with markup-like expressions in JavaScript. TSX combines TypeScript with JSX. Browsers run the transformed JavaScript, not raw TypeScript or JSX.

Our project mixes `.tsx` and `.jsx`. `allowJs: true` permits JavaScript files in the TypeScript project. JavaScript is not given the same full static checking automatically; `checkJs` is not enabled in our configuration.

Reference: TypeScript for JavaScript programmers.

05 / React concepts you need to explain

Component, props and state

A component is a reusable piece of UI expressed as a function. `Login`, `Landing` and `InvestigationFlow` are components. Props are inputs passed from a parent component. State is data a component remembers between renders.

Example: `Login` receives `mode` and `onSuccess` as props. Its email, password, visibility and status are local state. `onSuccess` is a callback that informs the parent that the demo login succeeded.

```
const [email, setEmail] = useState('');

<input
  value={email}
  onChange={event => setEmail(event.target.value)}
/>
```

This is a controlled input: React state supplies its value and an event updates that state. Typing changes the value, then React renders the UI using the new state.

Rendering and the DOM

Rendering means React calls components to determine the next UI. React then commits the required DOM changes. The DOM is the browser's object representation of the document. A render does not mean that the entire page is downloaded again or that every DOM element is replaced.

The hooks in our source

Hook	Purpose here
`useState`	Form values, visible view, sign-in status, checklist and calculator assumptions.
`useEffect`	Register events, update titles, start timers and clean them up.
`useRef`	Hold timer handles, DOM elements and animation references without using state.

Call hooks at the top level of a component or custom hook, rather than conditionally inside an event handler. When an update depends on the previous state, use a functional update such as `setStage(s => s + 1)`.

Why cleanup matters

An effect that starts a timer or event listener should release it when appropriate. Otherwise, old work may continue after the screen changes. The investigation flow clears its timer on cleanup; the navigation component clears timers and listeners.

References: React hooks, useState, useEffect.

06 / Routing, state and the user journey

How our navigation works

`main.tsx` reads `window.location.hash` and listens for `hashchange`. It conditionally renders the matching component. This is a simple custom hash router. React Router is not installed in the active replacement frontend.

Changing `#home` to `#login` changes the visible view without requesting a separate HTML document for every page. That is a single-page application, or SPA. The server still serves the entry document and assets.

Where state belongs

- `Frontend` owns `signedIn` because it decides whether the workspace is available.
- `Login` owns transient input and animation state.
- `Investigation` owns the original order details.
- `InvestigationFlow` owns progress, manual product facts and the report view.
- The report owns editable assumptions, checklist choices, open dialogs and notices.

This is local React state and props. We do not need a global state library for this prototype. If shared data and cross-screen persistence grow, we would reassess the state model rather than add a library only for its name.

What happens on refresh?

React memory is recreated. Demo sign-in is lost, and opening `#analyze` without that state redirects to login. Form details, checklist progress and report edits are not durably stored. A JSON export is a downloaded snapshot; it does not make the app state persistent.

Why “route protection” is not real security here

Our sign-in flag controls which UI is shown. A person can inspect or modify frontend code. Real access control must happen on the server for every protected operation, using a verified session or other supported authentication mechanism.

Tradeoff to mention

Hash routing is small and convenient for a prototype. A larger product would benefit from a maintained router, explicit nested routes, predictable navigation state and tested deep links. If history-based routes are introduced, the host may need to return the entry document for those routes.

07 / How the paper-plane animation works

We use more than one animation mechanism

Framer Motion handles React-friendly transitions, motion values and springs. CSS keyframes handle decorative background motion. The Web Animations API, through `element.animate()`, controls some browser-native animation playback. SVG draws the plane; SVG itself is a graphics format, not the animation library.

Landing-to-login flight

`PageFlight.tsx` finds suitable login/sign-up links, measures their positions, and animates a plane before changing the hash. It uses a broad cubic Bezier curve: a start point, an end point and two control points determine the shape.

The code samples 601 positions on that curve, estimates distance along it, and uses a binary search to find the position for the current animation progress. This makes progress track travelled distance more closely than advancing the raw curve parameter alone. Easing still deliberately changes speed near takeoff and arrival.

The plane's heading follows the curve's tangent, estimated from nearby points. Angle unwrapping avoids a sudden turn when angles cross the -180/180-degree boundary. A small fixed correction aligns the SVG artwork's nose with the direction of movement.

Login background flight

`Login.jsx` builds varying smooth routes and assigns a CSS `offset-path`. The browser follows that path with `offset-distance`. Playback rate slows when the form is focused. Motion springs smooth mouse parallax. This is separate from the landing-page navigation flight.

Performance and accessibility answer

Transforms and opacity are generally good starting points for animation performance. Blur, shadows, grain and path animation still have costs, so profile on real devices. `will-change` is a hint, not a guarantee of GPU acceleration or 60 fps.

Reduced-motion preferences are respected, and decorative artwork is hidden from assistive technology. CSS `shape()` motion paths need browser-compatibility checks; do not claim universal support or measured frame rates without evidence.

Reference: Motion for React.

08 / Forms, accessibility and responsive design

Validation in the current form

HTML constraints enforce required fields, email/URL shape, minimum quantity and integer steps. JavaScript applies additional checks, including a public HTTP/HTTPS listing requirement and a non-empty destination city. Password confirmation is checked in the sign-up demo.

Client-side validation improves feedback. It can be bypassed and cannot replace server-side validation. A well-formed URL can still point to an unsuitable or malicious resource.

Important URL-security distinction

The frontend currently does not fetch submitted product URLs. When a backend eventually fetches them, it needs protection against server-side request forgery, or SSRF: tricking a server into requesting internal or restricted services. Rejecting `localhost` in a browser form is not complete protection. Server checks must account for DNS resolution, private ranges, redirects and fetch restrictions.

Accessibility choices already in the UI

- Labels are associated with inputs; password visibility buttons have accessible names.
- Error/status text is announced using live regions or status semantics.
- Modal evidence and clear-report dialogs use native `<dialog>` behaviour.
- Keyboard focus is moved to relevant headings after major view changes.
- Focus styling, a skip link, reduced motion and decorative `aria-hidden` graphics are present.

ARIA means Accessible Rich Internet Applications. It provides additional semantics to assistive technology. Prefer the correct native HTML control first. `aria-label` alone cannot make an otherwise unusable interaction accessible.

Responsive is not the same as accessible

Responsive design adapts layout to available space, using Grid/Flexbox, flexible sizes and media queries. Accessibility also covers keyboard operation, reading order, names, contrast, zoom, motion and more.

We implemented accessibility features; we have not completed a formal WCAG audit. Avoid saying “fully WCAG AA compliant” just because the palette looks readable. Audit contrast and all meaningful states before making that claim.

Reference: OWASP SSRF prevention.

09 / Investigation and report: explain the logic

The current state machine

```
running -> manual input, if needed -> running -> report
```

The six stages are: read listing, identify product, find requirements, check evidence/claims, estimate landed cost, and prepare the report. A timeout advances the current stage. An interval updates elapsed time. These represent the intended interaction, not an actual research job.

For the Bluetooth-speaker sample URL, illustrative product facts are available locally. A custom URL pauses after the first stage and asks for product details. Cancel returns to the retained original form. Completing the stages creates an in-memory report.

Why the report says “insufficient information”

No live product source or official record has been retrieved. We keep product facts, user inputs, certificate numbers and cost assumptions visible, while avoiding an unsupported importability judgment. A link to an authority's home page is a research starting point, not evidence of a completed check.

The calculator, using simplified notation

```
goods = quantity x unit_price_USD x INR_per_USD
base = goods + converted_freight
duty = base x duty_rate / 100
surcharge = duty x surcharge_rate / 100
tax = (base + duty + surcharge) x tax_rate / 100
total = base + duty + surcharge + tax + other_fees
per_unit = total / quantity
```

These are the implemented illustrative formulas, not an assertion of the applicable customs valuation or tax treatment. The low/high scenarios are 90% and 115% of the current total, not independently researched quotes. Missing freight is treated as zero and exclusions are disclosed.

Browser capabilities in the report

Clipboard copy uses `navigator.clipboard.writeText()`. JSON export serialises a snapshot into a `Blob`, creates a temporary object URL, triggers a download and later revokes the URL. Print/PDF calls `window.print()` and uses print CSS; saving a PDF depends on the browser's print options.

10 / APIs, backend and TinyFish

What is an API?

An application programming interface defines how one part of a system interacts with another. A web API usually accepts HTTP requests and returns structured responses. The frontend handles interaction; a backend can handle protected operations, external services, jobs and persistent data.

Intended connection, not current execution

```
React form -> backend API -> investigation job -> external research / TinyFish -> evidence and checks -> report JSON -> React views
```

TinyFish is intended as a web-agent/retrieval layer in the product design. It is not a React UI library or a database. The active frontend contains no TinyFish SDK or research request. The backend integration and its actual service contract must be confirmed with the backend owner.

An illustrative future request

```
// Proposed integration example; not in the active frontend.
const response = await fetch('/api/investigations', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ url, quantity, destination })
});
if (!response.ok) throw new Error('Could not start investigation');
const result = await response.json();
```

The endpoint and payload above are examples, not a verified existing backend contract. `fetch` returns a Promise. `await` pauses that async function until the Promise settles; it does not freeze the whole browser. HTTP errors such as 404 do not automatically make `fetch` reject, so check the response status.

How should real progress reach the browser?

Polling repeatedly requests job status. Server-sent events, or SSE, stream updates from server to browser. WebSockets support two-way messages. Choose according to the actual needs; a research job does not automatically require WebSockets. Cancel should request server cancellation as well as update the UI.

Distinguish loading, success, failure, partial results, cancellation and user-input states. Preserve provenance and avoid showing a verified result before the service supplies evidence.

Reference: MDN Fetch API.

11 / JSON, YAML and .env: what are they?

These are data/configuration formats. They are not competing frontend frameworks.

JSON: JavaScript Object Notation

JSON represents objects, arrays, strings, numbers, booleans and null. It is common in API messages, package files and exported reports. Standard JSON uses double-quoted keys/strings and does not allow comments or trailing commas.

```
{
  "product": "Bluetooth speaker",
  "quantity": 100,
  "features": ["wireless", "battery"],
  "verified": false
}
```

`JSON.stringify(value)` creates JSON text. `JSON.parse(text)` turns valid JSON text into a JavaScript value. A JavaScript object in memory and a JSON string are related, but are not the same thing.

YAML: YAML Ain't Markup Language

YAML is a human-readable data format often used for CI workflows and deployment configuration. In its block style, indentation expresses nesting, `key: value` expresses a mapping, and `-` begins a sequence item. Use spaces for indentation, not tabs.

```
# The same example, written as YAML
product: Bluetooth speaker
quantity: 100
features:
  - wireless
  - battery
verified: false
```

YAML supports comments and more syntax than JSON, which also makes careless formatting easier to get wrong. A parser reads YAML; another tool gives those values a purpose. YAML by itself does not run a deployment or build a frontend.

.env: environment configuration

A `.env` file commonly contains `NAME=value` lines. In Vite, client-exposed `VITE_*` values are included in the built app. They must not hold secret API keys. An API base URL can be public configuration; a private service token belongs on the backend.

Our frontend uses JSON configuration files and JSON export. YAML is not required by the active frontend code; explain it as a related tooling format rather than claiming we used it to build the pages.

References: JSON, YAML specification, Vite environment variables.

12 / Node, npm, packages and configuration

Node.js versus the browser

Node.js is a JavaScript runtime used here to run development tools. The browser runs the shipped frontend JavaScript. Node is not the UI framework, and the Vite dev server is not the production research backend.

Files the judges may point at

Term or file	Plain meaning
`package.json`	Declares project metadata, scripts and dependency ranges.
`dependencies`	Packages used by the application, such as React and Framer Motion.
`devDependencies`	Build/checking tools such as Vite, TypeScript and type definitions.
`package-lock.json`	Captures resolved packages for more repeatable installs. Commit it with application code.
`node_modules`	Locally installed package files. Regenerate from the manifest/lockfile.
`.gitignore`	Tells Git which untracked files to ignore; it does not remove already tracked secrets.
`tsconfig.json`	Controls TypeScript checking and module/JSX handling.
`strict: true`	Enables a stricter family of TypeScript checks.
`noEmit: true`	The TypeScript checker writes no JavaScript; Vite handles the production output.

Commands to recognise

```
cd 'C:\PC\Projects\IIITD\import safe\frontend'
npm ci
npm run dev -- --port 5192 --strictPort
npm run build
npm run preview -- --port 5193
```

Run commands from the project directory. `npm ci` installs from the lockfile and expects it to agree with the manifest; it replaces the installed dependency tree. `npm install` is used when installing/updating dependencies and can update the lockfile. Do not casually run installs during a live demo.

Our build script is `tsc --noEmit && vite build`: type checking must pass before the build starts. `npm run preview` serves the build locally for review; it is not a complete production hosting plan.

Version shorthand: for these nonzero-major versions, `^19.0.0` admits compatible releases below 20.0.0; `~5.9.3` admits patch updates below 5.10.0. Read the lockfile for the resolved version.

Reference: npm lockfile documentation.

13 / Security terms and honest answers

Authentication versus authorisation

Authentication establishes who a user is. Authorisation decides what that user may do. The current `signedIn` boolean demonstrates navigation; it establishes neither a verified identity nor server-enforced permissions.

For production, choose a supported authentication approach, verify credentials on the server, protect sessions, enforce permissions on every protected request, and plan logout and expiry. Passwords need appropriate server-side hashing, not a hard-coded comparison in shipped JavaScript. A JWT is a token format, not proof that the whole system is secure; we do not implement JWT authentication here.

HTTPS, origins and CORS

HTTPS protects HTTP traffic using TLS. An origin is the combination of scheme, hostname and port. `http://localhost:5192` and `http://localhost:8000` are different origins even though the hostname matches.

CORS is a browser mechanism that lets a server specify which origins may read certain cross-origin responses. It is not authentication and does not stop all non-browser clients from calling an API. Backend CORS configuration must match the actual frontend origin when that integration is added.

XSS and CSRF

Cross-site scripting, or XSS, means executing attacker-controlled code in the page context. React escapes ordinary text values, but unsafe HTML injection, unsafe URLs and other sinks still need care. Do not inject untrusted seller-page HTML into the report.

Cross-site request forgery, or CSRF, tricks a browser into making an unwanted request using ambient credentials such as cookies. Its prevention depends on the authentication design. Secure cookie settings, origin checks and anti-CSRF measures may be relevant.

“Where do you store API keys?”

“Private research-service credentials belong on the backend, not in our React bundle, public Git history or `VITE_*` variables. Frontend code and bundled values can be inspected by the user.”

These are preparation topics and production requirements, not claims that all of these protections are implemented or audited in the prototype.

References: CORS, Vite environment exposure.

14 / Build, deployment, testing and debugging

Development versus production

During development, Vite serves modules locally and supports rapid update workflows. In production, the app's built HTML, JavaScript, CSS and assets can be hosted as static files. A separately running backend is needed for protected API operations and research jobs.

A successful build catches TypeScript/build problems and creates `dist/`. It does not prove that reports are accurate, every browser is supported, or the app is secure. Our active package scripts do not include an automated test command, and no automated test suite was added to this frontend.

What has been checked in this work

The replacement compiled successfully in the requested folder. Browser checks during implementation covered the demo and manual-entry flow, cancellation, evidence-dialog navigation, checklist changes and cost updates. Export and print actions are implemented, but do not describe this as exhaustive cross-browser or security testing.

Useful next tests

- Unit tests for calculator arithmetic, currency assumptions, missing prices and extreme inputs.
- Component/integration tests for invalid forms, manual fallback, cancel and stale timers.
- End-to-end tests for landing -> auth demo -> form -> report -> export.
- Keyboard, screen-reader, contrast, zoom and reduced-motion checks.
- Service-contract tests once the backend is connected; include failed and partial research.

Debug in the right order

1. Reproduce the issue and write the exact trigger.
2. Read the browser console for runtime errors.
3. Inspect the current state and DOM in browser tools.
4. For integrated services, inspect the Network panel and response status/body.
5. Check server logs when the failure is on the backend.
6. Fix the cause and reproduce the original trigger again.

Performance terms

Bundling packages code for delivery. Minification reduces code size. Tree shaking can remove unused module exports when supported. Lazy loading defers code or assets until needed; the current entry imports pages eagerly. HMR means hot module replacement; React Fast Refresh is a React-specific development integration, and should not be claimed as configured simply because Vite is installed.

15 / Interview answers: fundamentals

1. What is your frontend stack?

“React and React DOM, Vite, TypeScript plus one JavaScript/JSX view, custom CSS, SVG, and Framer Motion. Node.js and npm run the tooling. We use a small custom hash router and local React state.”

2. Why did you choose React?

“Our interface has reusable views and many state changes: inputs, progress, dialogs and report assumptions. React lets us express the UI as components driven by state.” Follow-up: reusable components improve structure; React alone does not guarantee speed or accessibility.

3. What does Vite do? Is it the backend?

“Vite is our development/build tool. It serves the app locally and builds deployable assets. Our research backend is a separate service; Vite is not that service.”

4. Why TypeScript if JavaScript already works?

“Types help us catch mismatched props and data shapes while developing. For example, an order declares its fields. TypeScript does not validate a network payload or replace runtime error handling.”

5. What are props versus state?

“Props are inputs a parent passes to a component. State is data that component remembers. Our Login gets its mode and success callback as props, and keeps email, password and status as state.”

6. What is YAML? Did you use it here?

“YAML is a readable data/configuration format often used by CI and deployment tools. It uses indentation for nesting in block style. Our frontend uses JSON config files; YAML is related tooling knowledge, not a required technology in this implementation.”

7. What is JSON?

“JSON is a structured text format for data exchange. We use it for package configuration and report export. An API would also commonly use JSON for requests and responses.”

8. Why custom CSS instead of a UI library?

“We wanted control over the sage/cream visual identity, glass card and paper-plane effects. The tradeoff is that we own component consistency, accessibility checks and responsive behaviour instead of inheriting a UI kit.”

16 / Interview answers: implementation

9. How does navigation work?

“The entry component reads the URL hash and listens for `hashchange`, then renders the matching view. It is a custom hash router, not React Router. Progress and report are internal states of the workspace flow.”

10. How did you make the plane turn smoothly?

“We replaced a path that doubled back sharply with a broad Bezier curve. We sample distance along it and align rotation to its tangent. Unwrapping the angle prevents jumps at the angle boundary.”

11. How does reduced motion work?

“The UI reads the user's reduced-motion preference and reduces or skips decorative movement. We also hide decorative graphics from screen readers and keep the main form usable without the animation.”

12. Does the progress bar show real backend progress?

“Currently it demonstrates the workflow with timed stages. For production, it should render backend job state from polling or a stream. We label the current flow as a demo.”

13. What happens when a listing cannot be read?

“The current custom-listing path explains that live access is unavailable and asks for product details. The intended backend integration should distinguish blocked pages, timeouts and missing fields, then offer a useful manual fallback.”

14. How do report edits update the cost?

“Calculator assumptions are React state. We derive the totals from those values on render, so changing a unit price updates the expected scenario, breakdown and per-unit estimate. We label the assumptions as illustrative.”

15. How does your evidence drawer work?

“Evidence chips select a local entry and open a native modal dialog. Entries show what the input is, its origin and capture time. They also state that no external source was retrieved.”

16. Can you call this a complete importability engine?

“No. It is a frontend prototype of the decision-support workflow. Reliable findings require backend research, current rules, product matching, evidence provenance and a validated evaluation method.”

17 / Interview answers: integration and tradeoffs

17. Is your login secure?

“It is a demo navigation gate, not production authentication. The password comparison and sign-in state are in the browser. Production needs server-verified credentials, sessions, expiry and authorisation.”

18. Where is TinyFish used?

“It is part of our intended research-service architecture. The active frontend does not call it. The backend should hold service credentials, orchestrate retrieval and return structured, attributable results to the UI.”

19. Why should API keys stay off the frontend?

“The browser receives the frontend code and bundled configuration. Users can inspect them. Vite client variables are public configuration, so private service keys should stay on the server.”

20. What does CORS mean?

“CORS controls browser access to cross-origin responses through server headers. Different localhost ports are different origins. It is separate from authentication and cannot replace permission checks.”

21. What would you change for production?

“Connect the real API contract, use server authentication, persist investigations, render service-driven progress, validate returned data, strengthen error handling, and test security, accessibility and browser support.” Prioritise the backend connection and trustworthy evidence before adding more decorative animation.

22. Is it a progressive web app or server-rendered app?

“It is currently a client-rendered SPA. We have not implemented a service worker, installable PWA setup or server-side rendering in this frontend.” Do not infer SSR, Next.js or offline support from using React.

23. How would you scale the frontend?

“Split large views into focused components, maintain typed service contracts, add appropriate tests, load heavy routes on demand, and separate server data from local UI state. We would profile before claiming a performance improvement.”

24. What is CI/CD and where does YAML fit?

“Continuous integration runs checks after code changes. Continuous delivery/deployment prepares or releases builds. A workflow can be described in YAML, but a CI platform interprets and executes it. We are not claiming a deployed CI pipeline for the current frontend.”

18 / Demo script and last-minute revision

A four-minute presentation

1. **0:00-0:40 / Explain the goal.** Open Home and give the 45-second introduction. Name the stack and state the demo boundary.
2. **0:40-1:15 / Show the interaction.** Click Log in or Get started. Explain the paper-plane transition, custom CSS and reduced-motion support. Use the demo login.
3. **1:15-2:00 / Show the form.** Use “Try the demo”, point out the fields, and explain client validation. Start the demo investigation.
4. **2:00-3:00 / Show the report.** Explain the six timed stages, insufficient-information outcome, product provenance and unverified checks.
5. **3:00-4:00 / Show useful behaviour.** Edit the unit price; show recalculation, an evidence entry, checklist, seller-message copy and JSON export. Explain the next backend integration step.

Questions to settle as a team before the round

- Which backend endpoints are actually running, and which frontend, if any, calls them?
- Does any teammate's backend call TinyFish? Can that teammate show the code and response?
- Which data is a fixture, which is user input, and which has real retrieved evidence?
- Has anything been deployed publicly, or is the demo only on localhost?
- Who owns explaining frontend, backend, domain checks and the integration boundary?

Ten facts worth remembering

1. React = UI library; Vite = dev/build tool; Node = runtime for our tooling.
2. TypeScript checks code; runtime validation checks incoming data.
3. Props come in; state is remembered; effects synchronise external work.
4. JSX/TSX become JavaScript before the browser runs them.
5. YAML and JSON are formats; `.env` is configuration.
6. A build succeeding is not the same as full testing.
7. Current sign-in and progress are demo behaviours.
8. Local component state does not survive a full reload.
9. CORS is not authentication; frontend keys are not private.
10. Evidence and current rules are needed before giving a confident import verdict.

If you do not know an answer, say what you do know, identify the boundary, and explain how you would verify it. Never invent an integration or benchmark.

19 / Fast glossary

Term	Meaning in one sentence
UI / UX	User interface / the user's experience of completing a task.
Component	A reusable unit of UI, such as a form or report section.
DOM	Browser objects representing the HTML document.
Event	A browser action or notification, such as a click or input change.
Callback	A function passed to another piece of code to invoke later.
Hook	A React function for state, effects, refs and other React features.
Ref	A persistent holder often used for a DOM node or timer handle.
SPA	A single-page application that changes views in the browser.
Route / hash	A view location / the fragment following # in our URLs.
API / endpoint	An interaction contract / a specific address in a web API.
HTTP / HTTPS	Web request-response protocol / HTTP protected with TLS.
GET / POST	Common HTTP methods for retrieval / submitting data or starting work.
2xx / 4xx / 5xx	Broad HTTP status groups for success / client-related error / server error.
Async / Promise	Work completing later / an object representing its eventual result.
JSON / YAML	Structured data formats; neither is a UI framework.
SDK	A software development kit that helps integrate a service or platform.
CLI	A command-line interface, such as npm commands in PowerShell.
Dependency	A package another project relies on.
Lockfile	A record of the resolved dependency tree.
Build / bundle	Produce deployable output / package modules for delivery.
HMR	Updating changed modules during development without always reloading everything.
CI/CD	Automated integration checks and delivery/deployment workflows.
SVG / Bezier	Vector graphics format / a curve shaped by control points.
Easing / spring	Timing of an animation / motion model that settles toward a target.
Media query	A CSS condition used to adapt styling.
ARIA	Extra accessibility semantics for assistive technologies.
Provenance	Where a fact came from and how it was obtained.
Mock / fixture	A substitute implementation / prepared example data.
Persistence	Data remaining available after the current process or page state ends.
CORS / SSRF	Browser cross-origin response access / abuse of server-side fetching.

20 / Related tools: recognise them without overclaiming

Git versus GitHub

Git is a version-control system that records changes. A repository, or repo, contains the project's tracked history. A commit is a recorded snapshot of tracked changes. A branch lets a line of work develop independently before it is merged.

GitHub hosts Git repositories and provides collaboration tools. A pull request proposes a change for review; Git itself does not require GitHub or pull requests. A merge combines lines of development. A conflict means the changes need a human decision before they can be combined safely.

Docker: image versus container

Docker packages software and its dependencies into images. A container is a running instance created from an image. This can help make development or deployment environments more consistent. A container is not the same as a full virtual machine.

Docker is not required by our active Vite frontend. Do not claim the app is containerised unless you can point to its Docker setup and show it running. A Dockerfile contains image-build instructions; Compose commonly uses YAML to describe multiple services.

CI/CD: what a YAML workflow might describe

Conceptually, a frontend pipeline could check out the repository, install the declared Node version, run `npm ci`, run checks, build the app and publish `dist/`. The YAML describes the workflow; the CI runner executes the commands. No pipeline is claimed as configured here.

Other words that often come up

Term	What to say
Library / framework	A library supplies reusable capabilities; a framework supplies broader application conventions. React is described as a UI library.
CSR / SSR	Client-side rendering builds the UI in the browser; server-side rendering produces initial HTML on a server. Our app uses CSR.
CDN	A content delivery network serves assets from distributed locations. Not a service we have configured in this frontend.
REST	An architectural style for resource-oriented interfaces; using JSON alone does not establish that an API is RESTful.
Cookie / localStorage	Browser storage mechanisms with different access and request behaviour. Neither makes data private from all page code by default.
localhost / port	Your own machine / the numbered endpoint used by a service, such as 5192 for this dev preview.

References: Git basics, Docker overview.

21 / Sources and code-reading checklist

Primary source for project-specific claims

The active frontend under `C:\PC\Projects\IIITD\import safe\frontend` was reviewed for this guide. It includes the manifest, lockfile, TypeScript config, entry point, public views, login, form, navigation animation and investigation/report flow. The former frontend backup is not the stack documented here.

The code snapshot is dated 24 October 2026. If the app changes, recheck demo authentication, routing, timers, API calls, persistence, dependencies and deployment before using these answers.

Official learning references

1. React: describing the UI - components, props and JSX.
2. React API reference - the React feature index.
3. React useState - component state.
4. React useEffect - effects and cleanup.
5. TypeScript introduction - types and JavaScript.
6. Vite guide - development and production builds.
7. Vite environment variables - client exposure and configuration.
8. Motion for React - motion components and values. Our installed package is named `framer-motion`; current docs may show newer import conventions.
9. npm lockfile documentation - resolved package information.
10. YAML 1.2.2 specification - mappings, sequences and indentation.
11. MDN JSON reference - JSON parsing and serialisation.
12. MDN CORS guide - origins and browser access to responses.
13. MDN Fetch API - HTTP requests and error handling.
14. OWASP SSRF prevention - safe server-side fetching.
15. Git basics - version control.
16. Docker overview - images and containers.

These links are for learning technical concepts. They are not regulatory evidence for a product import decision. Documentation sites can describe newer versions than the app's installed packages; follow our actual manifest and lockfile when explaining this implementation.

Files to open while preparing

- Open `package.json` and explain every dependency and script.
- Open `main.tsx` and explain the route and demo sign-in state.
- Open `Login.jsx` and identify the demo password check.
- Open `PageFlight.tsx` and point out the Bezier samples and tangent rotation.
- Open `InvestigationFlow.tsx` and distinguish the timer, manual fallback, calculator and exported data.

Prepared as a study aid. Understand each answer, then explain it naturally instead of memorising terminology without the code behind it.